

ENGINEERING ELECTIVE • CLOUD
TRACK

Linux Permissions, Users & Shell

Ownership, Identity &
Environment Variables

~2h 30m / Lecture by: Imran Sayyed

```
student@ubuntu:~$ ls -l
report.txt
-rw-r--r-- 1 imran dev
report.txt

student@ubuntu:~$ whoami
imran

student@ubuntu:~$ echo $HOME
/home/imran
```

What we cover today

01

Permissions

rwX, numbers, chmod

02

Ownership

chown, chgrp, users

03

Users & sudo

root, identity, access

04

Shell & Env

PATH, HOME, export

Every cloud server is multi-user. These four topics keep it secure and predictable.

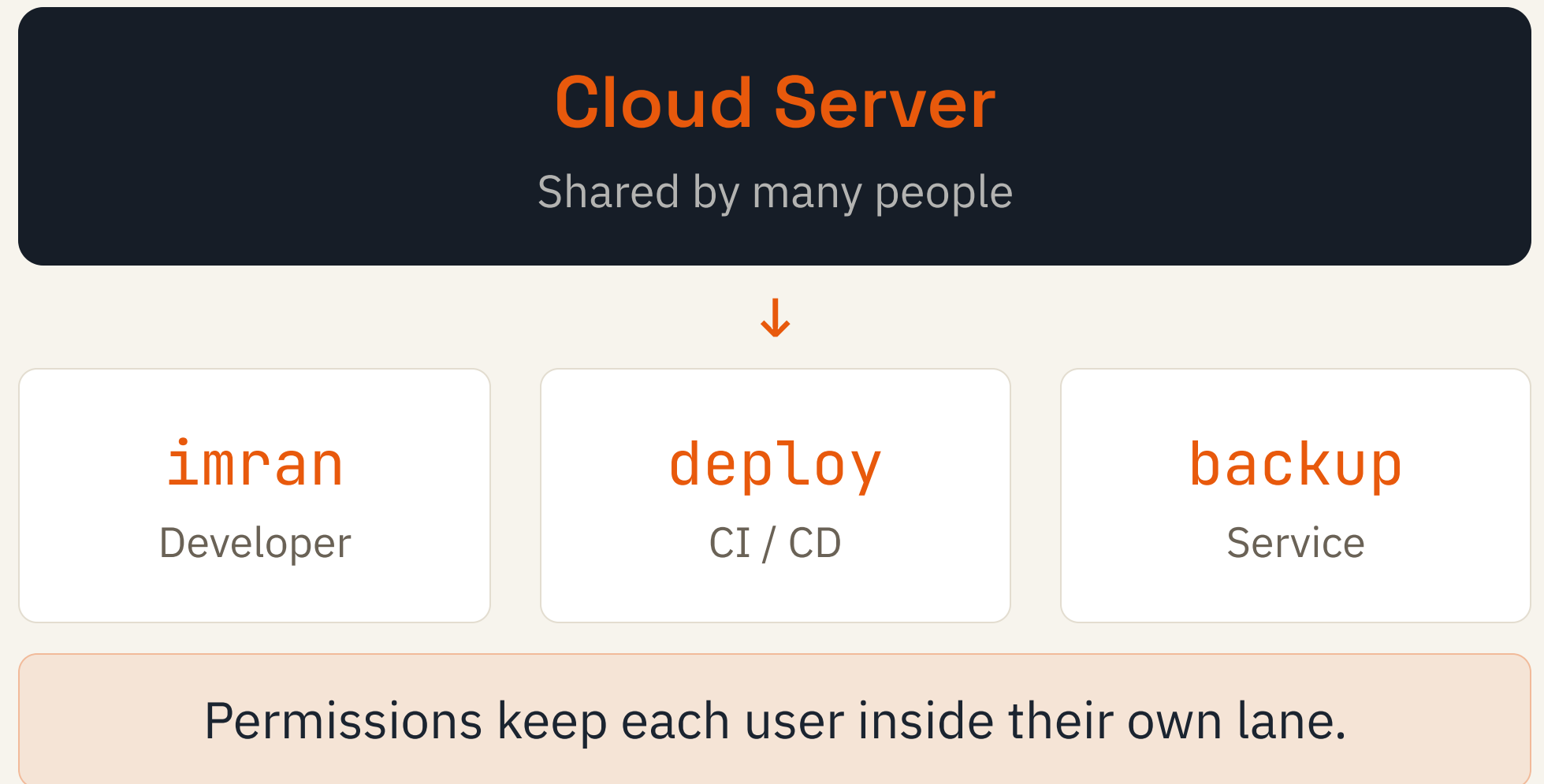
THE MOTIVATION

Why Linux security matters

One server, many users.

Permissions decide who can read, change, or run each file.

- ▶ Protects private data from other users
- ▶ Stops accidental changes to the system



PART ONE

Permissions & Ownership

Who can read, write, and run each file — and who that file belongs to.

THE CORE IDEA

The Linux permission model

OWNER

The user

The person who created the
file

GROUP

The team

A set of users sharing access

OTHERS

Everyone else

All other users on the system

Every file stores three separate permission sets — one for each group above.

THE THREE PERMISSIONS

Understanding r, w, x

r

Read

View the file's contents or
list a folder

w

Write

Edit the file or add and
remove items in a folder

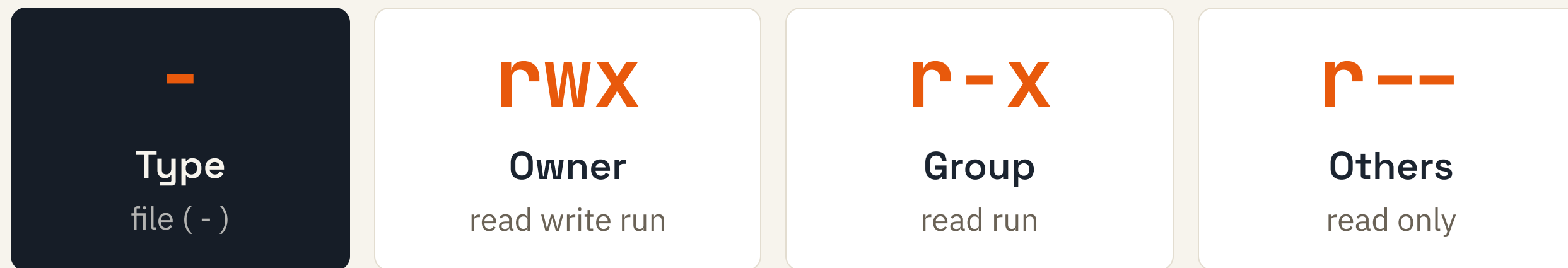
x

Execute

Run the file as a program or
enter a folder

A dash - in place of a letter means that permission is switched off.

Breaking down -rwxr-xr--



First character is the type. The next nine are three sets of rwx — owner, group, others.

Numeric permissions

THE VALUES

4 read (r)

2 write (w)

1 execute (x)

Add them up per group: $4 + 2 + 1 = 7$.

777 Everyone can do everything

755 Owner full, others read & run

644 Owner edits, others read only

600 Only the owner can read & write

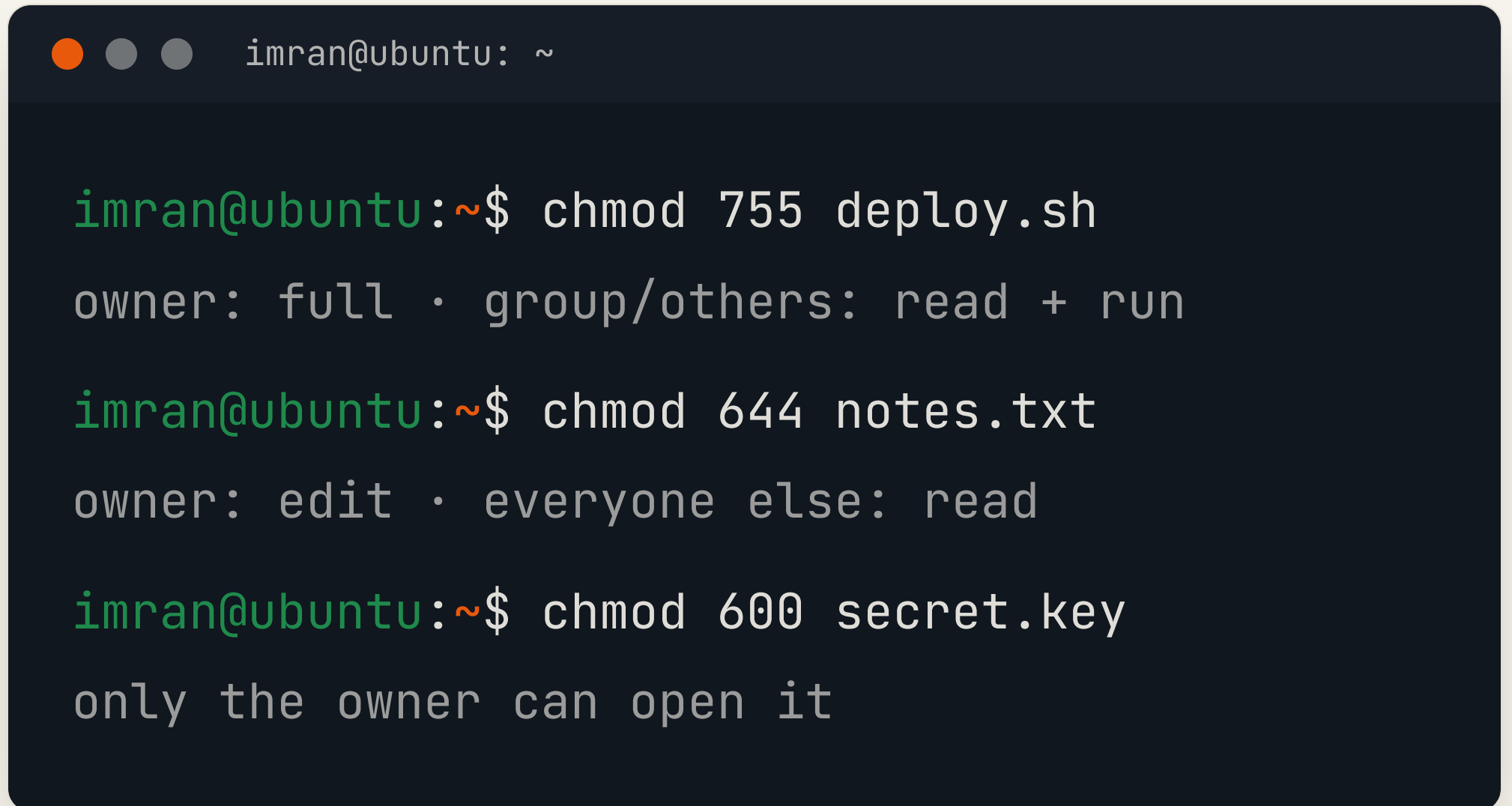
CHANGING PERMISSIONS

The chmod command

chmod sets permissions using the numbers you just learned.

PATTERN

```
chmod <number> <file>
```

A terminal window with a dark background and light text. The window title is 'imran@ubuntu: ~'. It shows three commands being executed: 'chmod 755 deploy.sh', 'chmod 644 notes.txt', and 'chmod 600 secret.key'. Each command is followed by a line explaining the permissions in a human-readable format: 'owner: full · group/others: read + run', 'owner: edit · everyone else: read', and 'only the owner can open it' respectively.

```
imran@ubuntu: ~$ chmod 755 deploy.sh
owner: full · group/others: read + run

imran@ubuntu: ~$ chmod 644 notes.txt
owner: edit · everyone else: read

imran@ubuntu: ~$ chmod 600 secret.key
only the owner can open it
```

Why chmod 777 is dangerous

```
imran@ubuntu:~$ chmod 777
```

```
app.py
```

```
rwXrwxrwx
```

Every user on the server can now read, edit, and run this file.

Anyone can edit it

Malicious code can be slipped in

Anyone can delete it

No protection from other accounts

Use 755 or 644 instead

Grant only the access actually needed

WHO OWNS WHAT

Ownership concepts

File

report.txt



Owner

imran



Group

dev



Others

everyone else

Every file carries both an owner and a group. Linux checks them in order to decide your access.

SEE IT

```
ls -l report.txt
```

```
-rw-r----- imran dev report.txt
```

CHANGING THE OWNER

The chown command

chown hands a file to a new owner — and optionally a new group.

PATTERN

```
chown user:group <file>
```

root@ubuntu: ~

```
root@ubuntu:~# chown imran report.txt
```

imran is now the owner

```
root@ubuntu:~# chown imran:dev report.txt
```

owner imran, group dev

```
root@ubuntu:~# chown -R imran /app
```

-R applies to a whole folder

CHANGING THE GROUP

The chgrp command

chgrp moves a file into a different group so a team can share it.

PATTERN

```
chgrp <group> <file>
```

root@ubuntu: ~

```
root@ubuntu:~# chgrp dev report.txt  
group is now dev
```

```
root@ubuntu:~# chgrp -R dev /project  
whole project shared with dev
```

```
root@ubuntu:~# ls -l report.txt  
-rw-rw-r-- imran dev report.txt
```

PUTTING IT TOGETHER

Real-world permission scenarios

644

Web page

Server reads it, visitors never edit it.

755

Deploy script

Owner edits, everyone can run it.

600

SSH key

Private — only the owner may read it.

664

Team file

Group edits together, others read.

700

Private folder

Only the owner can open or enter it.

444

Read-only log

Everyone reads, nobody edits.

PART TWO

Users & Groups

Who you are on a Linux system, and
how to gain admin rights safely.

IDENTITY ON LINUX

What is a user?

A user is an account the system recognises. Each one has a name, a number, and its own home folder.

- ▶ Every process runs as some user
- ▶ Each user has a numeric UID

One Linux Server

imran

UID 1001
/home/imran

sara

UID 1002
/home/sara

deploy

UID 1003
/home/deploy

Separate accounts mean separate files and separate permissions.

The root user

UID 0

root

Total control



Regular users

Limited to their own files

Root is the administrator account. It bypasses every permission check on the system.

- ▶ Can read, edit, or delete any file
- ▶ Can install software and manage users
- ▶ Never log in as root for daily work

THREE KINDS OF ACCOUNT

User types

UID 0

Root

The superuser. Unlimited control over the whole system.

UID 1–999

System

Service accounts that run software like web servers and databases.

UID 1000+

Regular

Real people. Each has a login, a password, and a home folder.

WHO AM I?

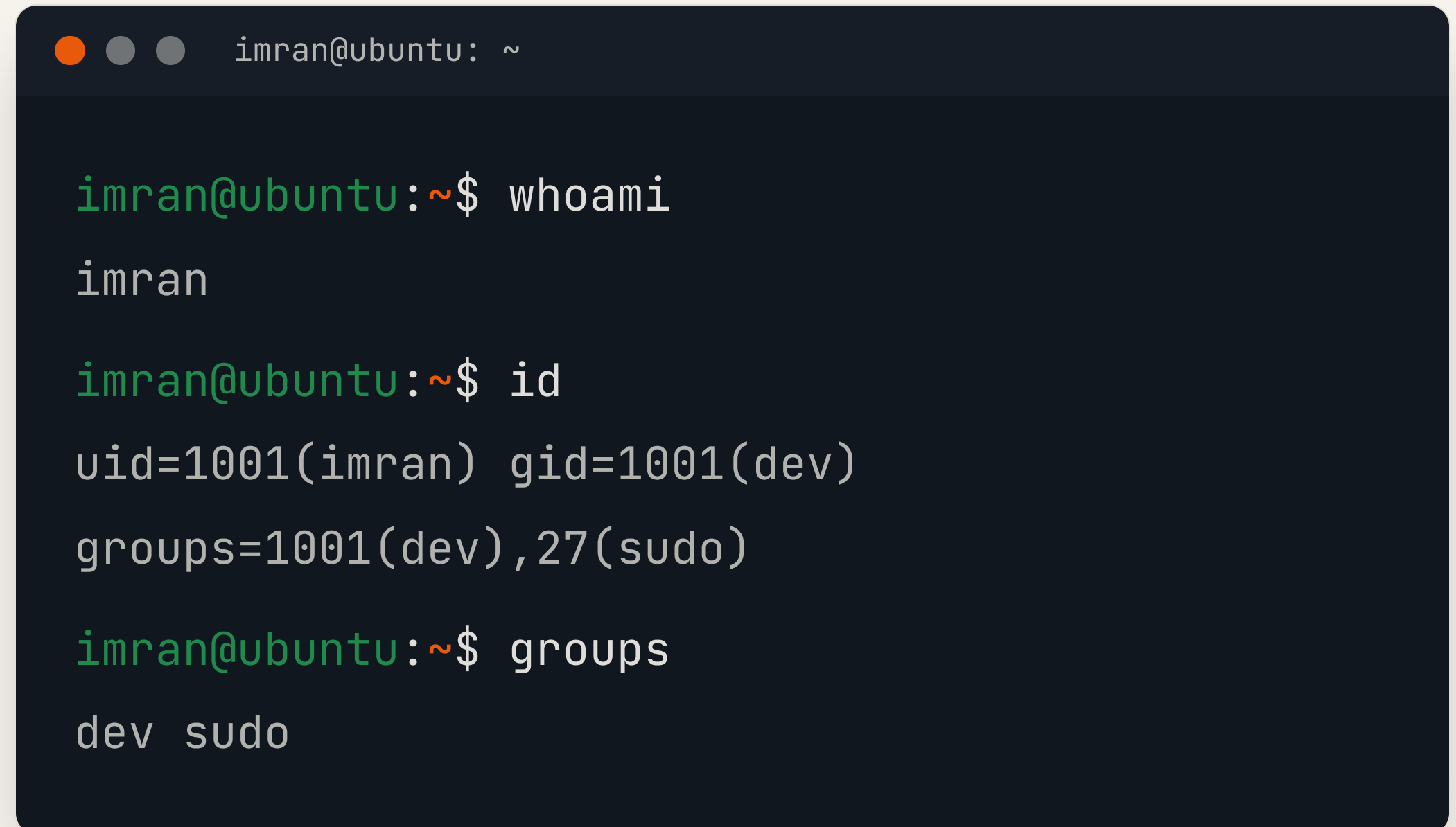
User identity commands

Three quick commands tell you exactly who you are and which groups you belong to.

whoami – your username

id – your UID and groups

groups – group names

A terminal window with a dark blue background and light gray text. The window title bar shows three colored circles (orange, gray, gray) followed by the text 'imran@ubuntu: ~'. The terminal content shows three commands and their outputs: 1. Command: 'whoami' (green prompt, orange tilde, green dollar sign). Output: 'imran'. 2. Command: 'id' (green prompt, orange tilde, green dollar sign). Output: 'uid=1001(imran) gid=1001(dev) groups=1001(dev),27(sudo)'. 3. Command: 'groups' (green prompt, orange tilde, green dollar sign). Output: 'dev sudo'.

```
imran@ubuntu: ~  
  
imran@ubuntu:~$ whoami  
imran  
  
imran@ubuntu:~$ id  
uid=1001(imran) gid=1001(dev)  
groups=1001(dev),27(sudo)  
  
imran@ubuntu:~$ groups  
dev sudo
```

The sudo command

Regular user

imran



sudo

Run one command as admin



Root privileges

Just for that command

sudo lets a trusted user run a single command as root, then drops back to normal.

```
imran@ubuntu:~$ sudo apt update
[sudo] password for imran:
imran@ubuntu:~$ sudo systemctl
restart nginx
```

PART THREE

Shell & Environment

The program that runs your commands, and the variables that shape how it behaves.

What is a shell?



The shell is the program that reads your typed commands and asks the kernel to carry them out.

FLAVOURS OF SHELL

Shell types

bash

Bourne Again

The default on most Linux servers. The one you will use most.

zsh

Z Shell

Bash-compatible with extra features. Default on modern macOS.

sh

Bourne Shell

The original, minimal shell. Used for portable scripts.

Check your current shell with `echo $SHELL`.

WHAT THE SHELL DOES

Shell responsibilities

Reads commands

Takes what you type and works out what you mean.

Finds programs

Searches the PATH to locate the command you asked for.

Expands variables

Replaces things like \$HOME with their value.

Runs scripts

Executes saved sequences of commands automatically.

EVERYDAY TOOLS

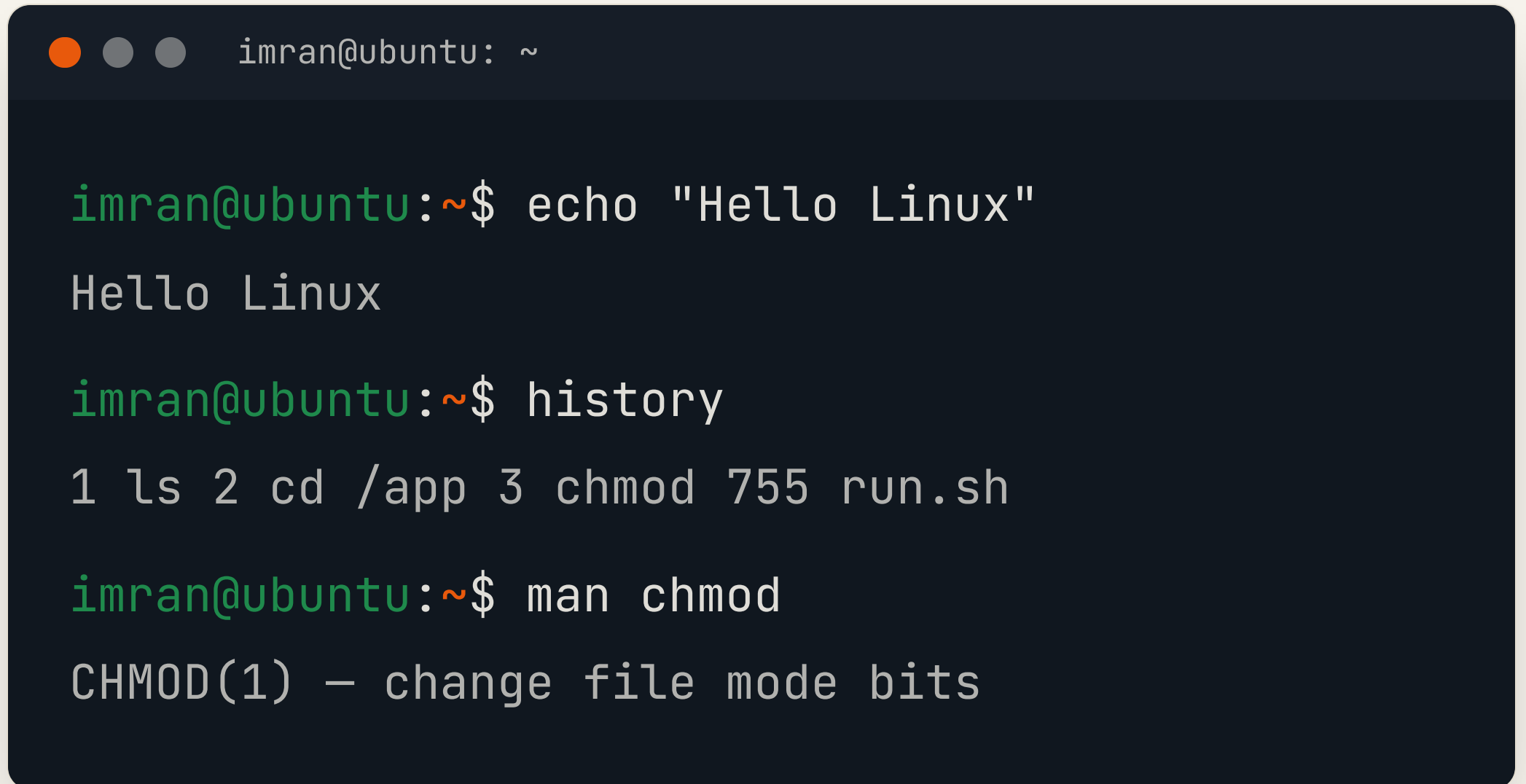
Useful shell commands

Three commands you will reach for in almost every session.

echo — print text or a variable

history — past commands

man — manual for any command

A terminal window with a dark background and light text. The window title bar shows three colored circles (orange, grey, grey) and the text 'imran@ubuntu: ~'. The terminal content shows three commands and their outputs: 1. 'echo "Hello Linux"' followed by 'Hello Linux'. 2. 'history' followed by '1 ls 2 cd /app 3 chmod 755 run.sh'. 3. 'man chmod' followed by 'CHMOD(1) — change file mode bits'.

```
imran@ubuntu: ~  
  
imran@ubuntu:~$ echo "Hello Linux"  
Hello Linux  
  
imran@ubuntu:~$ history  
1 ls 2 cd /app 3 chmod 755 run.sh  
  
imran@ubuntu:~$ man chmod  
CHMOD(1) — change file mode bits
```

Environment variables

User

logs in



Shell

loads the environment



Variables

PATH • HOME • SHELL • USER

Environment variables are named values the shell keeps in memory. Programs read them to know how to behave.

- ▷ Written in CAPITALS by convention
- ▷ Read with a \$ in front of the name

The PATH variable

You type

`ls`



Shell checks PATH

a list of folders



`/usr/bin/ls`

found here



Command runs

PATH is a list of folders the shell searches, in order, to find the program behind a command name.

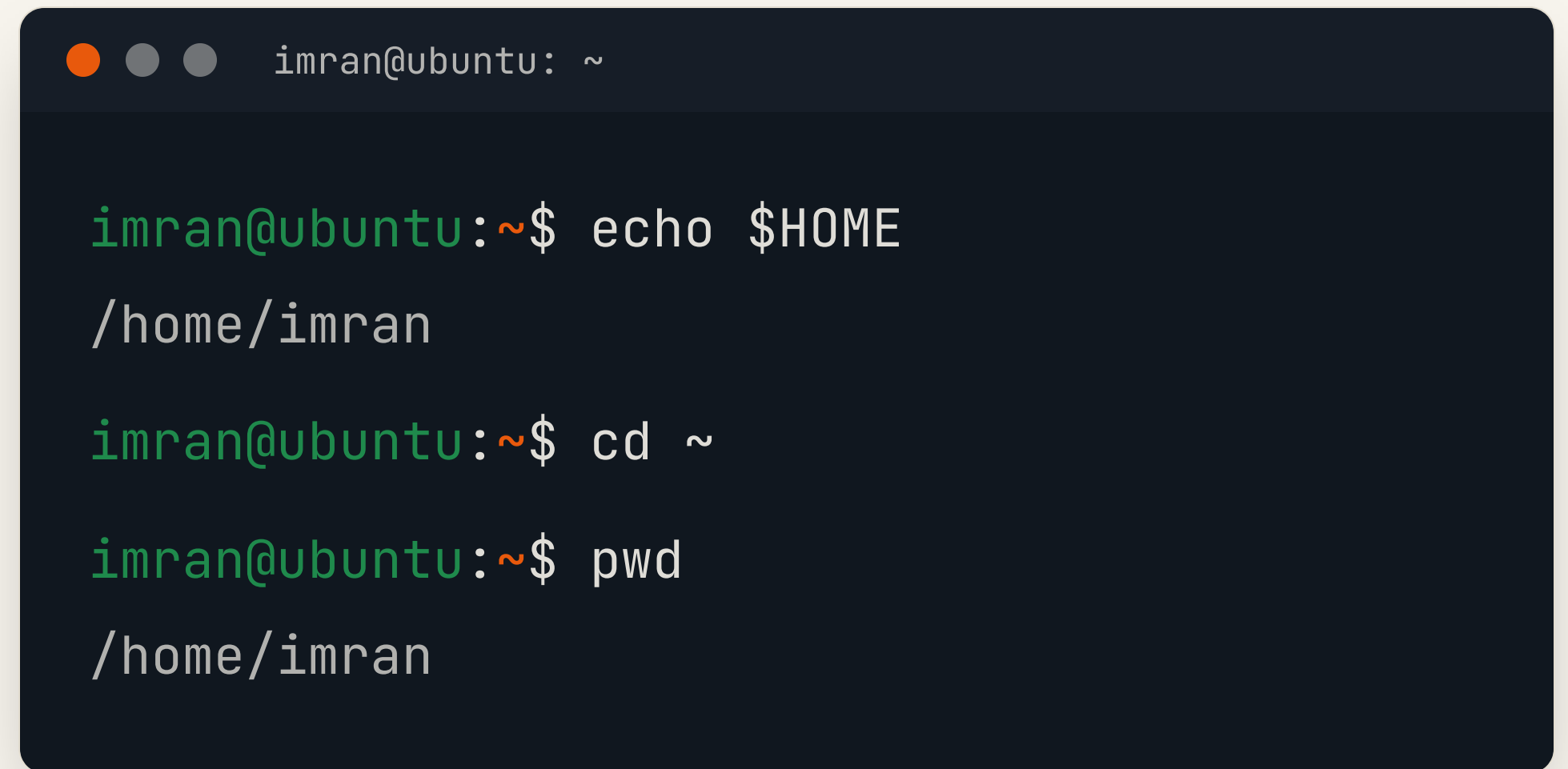
```
imran@ubuntu:~$ echo $PATH
/usr/local/bin:/usr/bin:/bin
```

YOUR PERSONAL FOLDER

The HOME variable

HOME points to your own folder — where your files and settings live.

- ▶ The shortcut ~ also means HOME
- ▶ Each user has a different HOME

A terminal window with a dark background and light-colored text. The window title bar shows three window control buttons (orange, grey, grey) and the text 'imran@ubuntu: ~'. The terminal content shows three commands and their outputs: 1. Command: 'echo \$HOME', Output: '/home/imran'. 2. Command: 'cd ~', Output: (no output shown). 3. Command: 'pwd', Output: '/home/imran'.

```
imran@ubuntu: ~  
  
imran@ubuntu:~$ echo $HOME  
/home/imran  
  
imran@ubuntu:~$ cd ~  
  
imran@ubuntu:~$ pwd  
/home/imran
```

READING VALUES

Displaying variables

Use **echo \$NAME** to read one variable, or **env** to list them all.

echo \$VAR – one value

env – every variable

printenv – same, by name

```
imran@ubuntu: ~  
  
imran@ubuntu:~$ echo $USER  
imran  
  
imran@ubuntu:~$ echo $SHELL  
/bin/bash  
  
imran@ubuntu:~$ printenv HOME  
/home/imran
```

SETTING VALUES

Creating variables

Use **export NAME=value** so the variable is available to programs you run.

- ▶ No spaces around the =
- ▶ Lasts until you close the shell

```
imran@ubuntu: ~  
  
imran@ubuntu:~$ export NAME=Imran  
  
imran@ubuntu:~$ echo $NAME  
Imran  
  
imran@ubuntu:~$ export ENV=production  
  
imran@ubuntu:~$ echo $ENV  
production
```

H A N D S - O N L A B

Secure a deploy script

Create a script, make it executable, and confirm who owns it — step by step in your terminal.

G O A L

A script only you can edit, but anyone can run.

```
$ touch deploy.sh
```

```
$ chmod 755 deploy.sh
```

```
$ ls -l deploy.sh
```

```
-rwxr-xr-x 1 imran dev 0 deploy.sh
```

```
$ echo $USER
```

```
imran
```

```
Check: owner can read, write, execute – others can  
read and execute only.
```

WHEN THINGS BREAK

Permission denied?

The two errors you will meet most often, and the first thing to check for each.

Permission denied

You lack rights on the file. Check with `ls -l`, then fix with `chmod` or use `sudo`.

command not found

The shell could not find it in `PATH`. Check spelling, or confirm the folder is in `echo $PATH`.

WHY THIS MATTERS IN THE CLOUD

On real servers

SSH keys

Private keys must be 600 — readable only by you, or SSH refuses them.

Service users

Apps run as limited users, never root, to contain any breach.

Env config

Secrets and settings reach apps through environment variables.

TEST YOURSELF

Pop quiz

Q1 What does the permission number 755 allow?

Q2 Which command changes who owns a file?

Q3 Why should you avoid running everything as root?

Q4 What does the PATH variable store?

Q5 How do you create an environment variable named ENV?

THAT'S A WRAP

Questions?

You can now read and set permissions, manage users with sudo, navigate the shell, and work with environment variables like PATH and HOME.

Remember

Least privilege always

Keep practising

Live in the terminal